

Submitted By:
Chaudry Ali Sher
232418
BSCSev-7-C

Parallel & Distributed Computing Lab - 4

Task 1:

```
Program.cs > C# Dev Tools > Program
1   using System;
2   using System.Threading;
3
    1 reference | 0 references | 0 references | 0 initializations | 0 initializations
4   class Program {
        4 references | 4 references | 0 references
5       const int NumThreads = 4;
        2 references | 2 references | 0 references
6       const int IncrementPerThread = 1_000_000;
7
        3 references | 3 references | 0 references
8       static long counter = 0;
9
        1 reference | 1 reference | 1 reference
10      static void Worker() {
11          for (int i = 0; i < IncrementPerThread; i++){
12              counter++;
13          }
14      }
15
        0 references | 0 references | 0 references
16      static void Main(){
```

```
        0 references | 0 references | 0 references
16      static void Main(){
17          Thread[] threads = new Thread[NumThreads];
18          for(int i = 0; i < NumThreads; i++){
19              threads[i] = new Thread(Worker);
20              threads[i].Start();
21          }
22          for(int i = 0; i < NumThreads; i++){
23              threads[i].Join();
24          }
25
26          long expected = (long)NumThreads * IncrementPerThread;
27          Console.WriteLine($"Expected: {expected}");
28          Console.WriteLine($"Actual: {counter}");
29          Console.WriteLine($"Lost Updates: {expected - counter}");
30          Console.WriteLine($"Logical Processors: {Environment.ProcessorCount}");
31      }
32  }
```

Results:

```
PS C:\Users\itsal\RaceLab> dotnet run
Expected: 4000000
Actual: 1222549
Lost Updates: 2777451
Logical Processors: 8
PS C:\Users\itsal\RaceLab> dotnet run
Expected: 4000000
Actual: 1468418
Lost Updates: 2531582
Logical Processors: 8
PS C:\Users\itsal\RaceLab> dotnet run
Expected: 4000000
Actual: 1071812
Lost Updates: 2928188
Logical Processors: 8
PS C:\Users\itsal\RaceLab> dotnet run
Expected: 4000000
Actual: 1878877
Lost Updates: 2121123
Logical Processors: 8
```

Results:

Run	Expected Value	Actual Value	Lost Updates
1	4000000	1222549	2777451
2	4000000	1468418	2531582
3	4000000	1071812	2928188
4	4000000	1878877	2121123
5	4000000	1380577	2619423
6	4000000	1800209	2199791
7	4000000	1318416	2681584
8	4000000	1354195	2645805
9	4000000	1125476	2874524
10	4000000	1302152	2697848

Task 2:

```
1  using System;
2  using System.Threading;
3
   0 references | 0 references | 0 initializations
4  class Program
5  {
       6 references | 0 references
6      const int NumThreads = 4;
       5 references | 0 references
7      const int IncrementsPerThread = 250_000;
8
       6 references | 0 references
9      static int counter = 0;
       1 reference | 0 references
10     static readonly object gate = new object();
       3 references | 0 references
11     static bool useLock = false;
12
       4 references | 0 references
13     static int[][] seenLog = new int[NumThreads][];
14
       1 reference | 1 reference
15     static void Worker(int id)
16     {
17         int[] log = seenLog[id];
18
19         for (int i = 0; i < IncrementsPerThread; i++)
20         {
21             if (useLock)
```

```
       1 reference | 1 reference
15     static void Worker(int id)
16     {
17         int[] log = seenLog[id];
18
19         for (int i = 0; i < IncrementsPerThread; i++)
20         {
21             if (useLock)
22             {
23                 lock (gate)
24                 {
25                     int seen = counter;
26                     log[i] = seen;
27                     counter = seen + 1;
28                 }
29             }
30             else
31             {
32                 int seen = counter;
33                 log[i] = seen;
34                 counter = seen + 1;
35             }
36         }
37     }
38
```

```
0 references | Add program arguments | 0 references
static void Main(string[] args)
{
    useLock = args.Length > 0 && args[0] == "--lock";

    int total = NumThreads * IncrementsPerThread;

    Thread[] threads = new Thread[NumThreads];

    for (int t = 0; t < NumThreads; t++)
    {
        seenLog[t] = new int[IncrementsPerThread];

        int id = t;

        threads[t] = new Thread(() => Worker(id));
        threads[t].Start();
    }

    foreach (Thread th in threads)
    {
        th.Join();
    }
```

```
    foreach (Thread th in threads)
    {
        th.Join();
    }

    // Count how many times each value was read
    int[] readCount = new int[total + 1];

    for (int t = 0; t < NumThreads; t++)
    {
        for (int i = 0; i < IncrementsPerThread; i++)
        {
            readCount[seenLog[t][i]]++;
        }
    }

    // Count collisions
    int collisions = 0;

    for (int v = 0; v <= total; v++)
    {
        if (readCount[v] > 1)
        {
            collisions += readCount[v] - 1;
        }
```

```
        // Count collisions
        int collisions = 0;

        for (int v = 0; v <= total; v++)
        {
            if (readCount[v] > 1)
            {
                collisions += readCount[v] - 1;
            }
        }

        string mode = useLock ? "with lock" : "no synchronization";

        Console.WriteLine($"Mode: {mode}");
        Console.WriteLine($"Total increments: {total}");
        Console.WriteLine($"Final counter: {counter}");
        Console.WriteLine($"Lost updates: {total - counter}");
        Console.WriteLine($"Collisions: {collisions}");

        // Print first 5 collisions
        int shown = 0;

        for (int v = 0; v <= total && shown < 5; v++)
        {
            if (readCount[v] > 1)
```

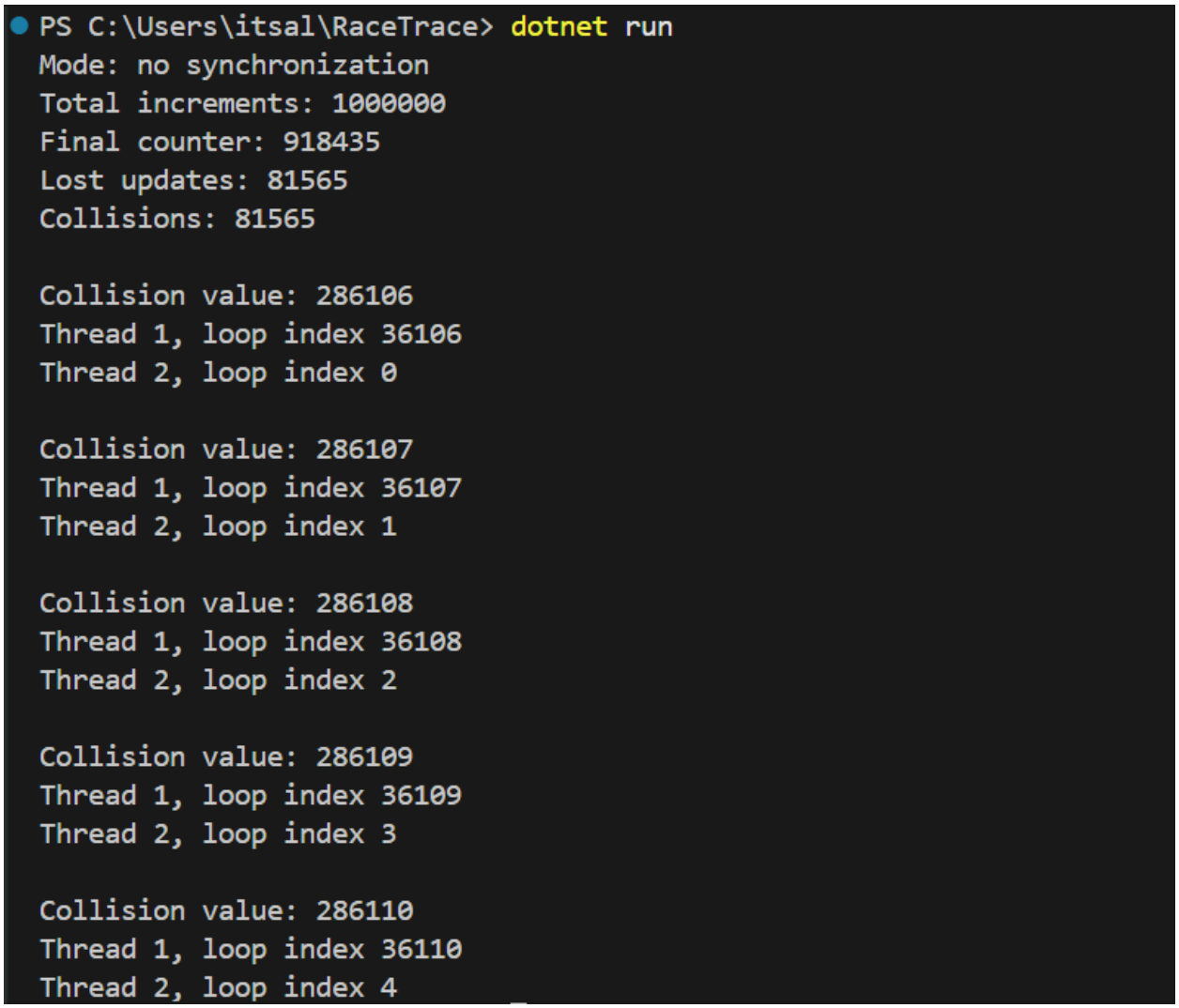
```
        // Print first 5 collisions
        int shown = 0;

        for (int v = 0; v <= total && shown < 5; v++)
        {
            if (readCount[v] > 1)
            {
                Console.WriteLine($"\nCollision value: {v}");

                for (int t = 0; t < NumThreads; t++)
                {
                    for (int i = 0; i < IncrementsPerThread; i++)
                    {
                        if (seenLog[t][i] == v)
                        {
                            Console.WriteLine(
                                $"Thread {t}, loop index {i}");
                        }
                    }
                }

                shown++;
            }
        }
    }
}
```

Results:


PS C:\Users\itsal\RaceTrace> dotnet run
Mode: no synchronization
Total increments: 1000000
Final counter: 918435
Lost updates: 81565
Collisions: 81565
Collision value: 286106
Thread 1, loop index 36106
Thread 2, loop index 0
Collision value: 286107
Thread 1, loop index 36107
Thread 2, loop index 1
Collision value: 286108
Thread 1, loop index 36108
Thread 2, loop index 2
Collision value: 286109
Thread 1, loop index 36109
Thread 2, loop index 3
Collision value: 286110
Thread 1, loop index 36110
Thread 2, loop index 4

Task 3:

```
using System;
using System.Threading;

0 references | 0 references | 0 initializations
class Program
{
    4 references | 0 references
    const int NumThreads = 4;
    2 references | 0 references
    const int IncrementsPerThread = 1_000_000;

    3 references | 0 references
    static long counter = 0;

    1 reference | 0 references
    static readonly object counterLock = new object();

    1 reference | 1 reference
    static void Worker()
    {
        for (int i = 0; i < IncrementsPerThread; i++)
        {
            lock (counterLock)
            {
                counter++;
            }
        }
    }

    0 references | Add program arguments | 0 references
    static void Main()
    {
        Thread[] threads = new Thread[NumThreads];

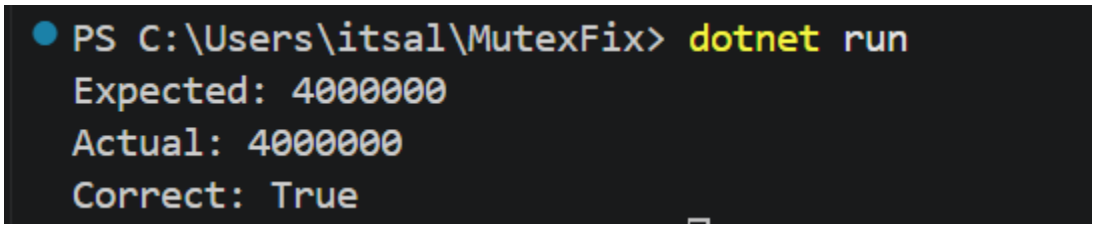
        for (int i = 0; i < NumThreads; i++)
        {
            threads[i] = new Thread(Worker);
            threads[i].Start();
        }

        for (int i = 0; i < NumThreads; i++)
        {
            threads[i].Join();
        }

        long expected =
            (long)NumThreads * IncrementsPerThread;

        Console.WriteLine($"Expected: {expected}");
        Console.WriteLine($"Actual: {counter}");
        Console.WriteLine($"Correct: {counter == expected}");
    }
}
```

Results:


PS C:\Users\itsal\MutexFix> dotnet run
Expected: 4000000
Actual: 4000000
Correct: True

Task 4:

```
using System;
using System.Diagnostics;
using System.Threading;

class Program
{
    const int NumThreads = 4;
    const int IncrementsPerThread = 1_000_000;

    static long counter = 0;
    static readonly object counterLock = new object();

    // Version 1: Using lock
    static void LockWorker()
    {
        for (int i = 0; i < IncrementsPerThread; i++)
        {
            lock (counterLock)
            {
                counter++;
            }
        }
    }

    // Version 2: Using atomic increment
    static void InterlockedWorker()
    {
        for (int i = 0; i < IncrementsPerThread; i++)
        {
            Interlocked.Increment(ref counter);
        }
    }
```

```
    // Version 2: Using atomic increment
    static void InterlockedWorker()
    {
        for (int i = 0; i < IncrementsPerThread; i++)
        {
            Interlocked.Increment(ref counter);
        }
    }

    static double RunVersion(string name, ThreadStart worker)
    {
        // Reset counter before each test
        counter = 0;

        Thread[] threads = new Thread[NumThreads];

        // Start measuring time
        Stopwatch sw = Stopwatch.StartNew();

        // Create and start threads
        for (int i = 0; i < NumThreads; i++)
        {
            threads[i] = new Thread(worker);
            threads[i].Start();
        }

        // Wait for all threads
        for (int i = 0; i < NumThreads; i++)
        {
            threads[i].Join();
        }
```

```
57          // Stop measuring time
58          sw.Stop();
59
60          long expected =
61              (long)NumThreads * IncrementsPerThread;
62
63          double ms = sw.Elapsed.TotalMilliseconds;
64
65          Console.WriteLine(
66              $"[{name}] expected={expected} " +
67              $"actual={counter} " +
68              $"correct={counter == expected} " +
69              $"time={ms:F2} ms");
70
71          return ms;
72      }
73
74      static void Main()
75      {
76          // Warm-up runs
77          RunVersion("warm-up lock", LockWorker);
78          RunVersion("warm-up Interlocked", InterlockedWorker);
79
80          // Actual measured runs
81          RunVersion("lock", LockWorker);
82          RunVersion("Interlocked.Increment", InterlockedWorker);
83      }
84  }
```

Results:

```
PS C:\Users\itsal\CounterBench> dotnet run
[warm-up lock] expected=4000000 actual=4000000 correct=True time=269.73 ms
[warm-up Interlocked] expected=4000000 actual=4000000 correct=True time=127.06 ms
[lock] expected=4000000 actual=4000000 correct=True time=288.50 ms
[Interlocked.Increment] expected=4000000 actual=4000000 correct=True time=93.79 ms
```